Dynamic Programming and Stochastic Control - Assignment 1

Zhenrui Zheng

Chinese University of Hong Kong, Shenzhen
225040512@link.cuhk.edu.cn

Contents

Problem 1: Dynamic Pricing in Discrete Time **1**
1.1 Derivation of Value Function 1
1.2 Proof of Monotonicity 2

Problem 2: Label correcting with negative arc lengths **4**
2.1 Proof of Correctness 4
2.2 Label Correcting Algorithm as a Special Case 6

Problem 3: Object Partition as Shortest Path Problem **7**

Problem 4: **9**

Problem 5: **10**

Problem 6: **11**

Problem 7: **12**

Problem 8: **13**

Problem 9: **14**

Problem 1: Dynamic Pricing in Discrete Time

1.1 Derivation of Value Function

Recall that in a Markov Decision Process with random state transition, the optimal action $a^*(s)$ in state s is defined as:

$$a^*(s) = \arg\max_{a \in A} Q(s,a) = \arg\max_{a \in A} \mathbb{E}_{s' \sim P(s'|s,a)} \left[r(s', s, a) + V(s') \right]$$

For the given problem, the state of kth day is the remaining unsold items x_k, and the action is the price u_k. Thus we can write the optimal action as:

$$u_k^{*1} = u^*(k, x_k) = \arg\max_{u \in \mathbb{R}_+} \mathbb{E}_{x_{k+1} \sim P(x_{k+1}|x_k,u)} \left[g(x_{k+1}, x_k, u_k) + V_{k+1}(x_{k+1}) \right]$$

Where $g(x_{k+1}, x_k, u_k)$ is the profit from selling $x_k - x_{k+1}$ items at price u, that is:

$$g(x_{k+1}, x_k, u_k) = u_k(x_k - x_{k+1})$$

and $V_{k+1}(x_{k+1})$ is the optimal expected value function of the next state $(k+1, x_{k+1})$. Since we already have:

$$q_k(u_k) = \alpha \exp(-u_k)$$

Thus the action value function $Q(x_k, u_k)$ for any $x_k > 0$ can be expanded (from its expectation form) as:

$$\begin{aligned}
Q(x_k, u_k) &= \alpha \exp(-u_k) \left[g(x_k - 1, x_k, u_k) + V_{k+1}(x_k - 1) \right] \\
&\quad + (1 - \alpha \exp(-u_k)) \left[g(x_k, x_k, u_k) + V_{k+1}(x_k) \right] \\
&= \alpha \exp(-u_k) \left[u_k + V_{k+1}(x_k - 1) \right] \\
&\quad + (1 - \alpha \exp(-u_k)) \left[V_{k+1}(x_k) \right]
\end{aligned}$$

To find the optimal u_k, we take the derivative of $Q(x_k, u_k)$ with respect to u_k and set it to zero:

$$\begin{aligned}
\frac{\partial Q}{\partial u_k} &= - \alpha \exp(-u_k) \left[u_k + V_{k+1}(x_k - 1) \right] \\
&\quad + \alpha \exp(-u_k) \cdot 1 \\
&\quad + \alpha \exp(-u_k) \left[V_{k+1}(x_k) \right] \\
&= \alpha \exp(-u_k) \left[1 - u_k - V_{k+1}(x_k - 1) + V_{k+1}(x_k) \right] = 0 \\
\implies & \alpha \exp(-u_k^*) \left[u_k^* + V_{k+1}(x_k - 1) - V_{k+1}(x_k) \right] = \alpha \exp(-u_k^*) \\
& u_k^* = 1 + V_{k+1}(x_k) - V_{k+1}(x_k - 1)
\end{aligned}$$

Thus,

$$\begin{aligned}
V_k(x_k) = Q(x_k, u_k^*) &= \alpha \exp(-u_k^*) \left[u_k^*(x_k) + V_{k+1}(x_k - 1) - V_{k+1}(x_k) \right] + V_{k+1}(x_k) \\
&= \alpha \exp(-u_k^*) + V_{k+1}(x_k)
\end{aligned}$$

[1] For simplicity, we use u_k instead of $u_k(x_k)$ when there is no ambiguity.

1.2 Proof of Monotonicity

Assume that $V_k(0) = 0$ and $V_N(x_N) = 0$, let's prove the closed form of $V_k(x_k)$ by induction. Assume that for some $k < N$, $V_k(x_k)$ has the closed form:

$$V_k(x_k) = \begin{cases} (N-k)\alpha\exp(-1) & \text{if } x_k \geq N-k \\ \sum_{i=k}^{N-x_k} \alpha\exp(-u_i^*(x_k)) + x_k\alpha\exp(-1) & \text{if } 0 < x_k < N-k \\ 0 & \text{if } x_k = 0 \end{cases}$$

For V_{k-1}, there are four cases:

1. $x_{k-1} = 0$
   $V_{k-1}(x_{k-1}) = 0$ according to our assumption.

2. $0 < x_{k-1} < N-k$

$$\begin{aligned} V_{k-1}(x_{k-1}) &= \alpha\exp(-u_{k-1}^*(x_{k-1})) + V_k(x_{k-1}) \\ &= \alpha\exp(-u_{k-1}^*(x_{k-1})) + \sum_{i=k}^{N-x_{k-1}} \alpha\exp(-u_i^*(x_{k-1})) + x_{k-1}\alpha\exp(-1) \\ &= \sum_{i=k-1}^{N-x_{k-1}} \alpha\exp(-u_i^*(x_{k-1})) + x_{k-1}\alpha\exp(-1) \end{aligned}$$

3. $x_{k-1} = N-k$
   Let's first derive $u_{k-1}^*(x_{k-1})$. Assume that for some $k < N-1$, $u_k^*(N-k-1) = 1$, then:

$$\begin{aligned} u_{k-1}^*(N-k) &= 1 + V_k(N-k) - V_k(N-k-1) \\ &= 1 + (N-k)\alpha\exp(-1) \\ &\quad - \left(\sum_{i=k}^{N-x_{k-1}+1} \alpha\exp(-u_i^*(x_{k-1}-1)) + (x_{k-1}-1)\alpha\exp(-1) \right) \\ &= 1 + (N-k)\alpha\exp(-1) - (\alpha\exp(-1) + (N-k-1)\alpha\exp(-1)) \\ &= 1 \end{aligned}$$

   Which also holds for $k = N-1$, $u_{N-1}^*(0) = 1$. Thus,

$$\begin{aligned} V_{k-1}(x_{k-1}) &= \alpha\exp(-u_{k-1}^*(x_{k-1})) + V_k(x_{k-1}) \\ &= \alpha\exp(-u_{k-1}^*(N-k)) + (N-k)\alpha\exp(-1) \\ &= \alpha\exp(-1) + (N-k)\alpha\exp(-1) \\ &= (N-(k-1))\alpha\exp(-1) \end{aligned}$$

4. $x_{k-1} \geq N-(k-1) \quad (x_{k-1} > N-k)$

$$\begin{aligned} u_{k-1}^*(x_{k-1}) &= 1 + V_k(x_{k-1}) - V_k(x_{k-1}-1) \\ &= 1 + (N-k)\alpha\exp(-1) - (N-k)\alpha\exp(-1) \quad (x_{k-1}-1 \geq N-k) \\ &= 1 \end{aligned}$$

   Thus,

$$\begin{aligned} V_{k-1}(x_{k-1}) &= \alpha\exp(-u_{k-1}^*(x_{k-1})) + V_k(x_{k-1}) \\ &= \alpha\exp(-1) + (N-k)\alpha\exp(-1) \\ &= (N-(k-1))\alpha\exp(-1) \end{aligned}$$

Therefore, we have proved that the induction relation holds. For $k = N$, It's trivial to prove that the assumption also holds by substitution. We have therefore proved the closed form of $V_k(x_k)$.

Then, we can prove the monotonically non-increasing property of $u_k^*(x_k)$.

$$u_k^*(x_k) = 1 + V_{k+1}(x_k) - V_{k+1}(x_k - 1)$$
$$u_k^*(x_k) - u_k^*(x_k - 1) = V_{k+1}(x_k) - 2V_{k+1}(x_k - 1) + V_{k+1}(x_k - 2)$$

Which actually requires us to prove that $V_{k+1}(x_k) - 2V_{k+1}(x_k - 1) + V_{k+1}(x_k - 2) \leq 0$, that is, V_k is a concave function. Since we have already derived the piecewise form of V_k, we can prove its concavity by case analysis on x_k, expanding the expressions of V_{k+1}, and using a little bit induction assumption. However, this would be quite tedious and the proof is straightforward to those who understand the problem structure.

Problem 2: Label correcting with negative arc lengths

2.1 Proof of Correctness

Let's prove that the modified algorithm terminates with a shortest path from s to t. The proof is divided into two parts: firstly, we prove that the algorithm must terminate; secondly, we prove that upon termination, the computed distance d_t is indeed the shortest path distance.

Part A: Termination

Let d_j be the label of node j, representing the length of a computed path from s to j. The algorithm initializes $d_s = 0$ and $d_j = \infty$ for all $j \neq s$.

Lemma 1. *For any node j, the label d_j is non-increasing throughout the execution of the algorithm.*

Proof. A label d_j is updated only in the Modified Step 2, where it is set to $d_i + a_{ij}$. The condition for this update is $d_i + a_{ij} < \min\{d_j, \text{UPPER} - u_j\}$, which implies $d_i + a_{ij} < d_j$. Thus, any update to d_j strictly decreases its value. □

Lemma 2. *The label d_j of any node j is always greater than or equal to the shortest path distance from s to j, denoted $\delta(s, j)$.*

Proof. We prove this by induction on the number of label updates.

Base Case: Initially, $d_s = 0 = \delta(s, s)$ and $d_j = \infty \geq \delta(s, j)$ for $j \neq s$, the property holds.

Induction: Assume that before any update, $d_k \geq \delta(s, k)$ for all nodes k. Consider an update to d_j where d_j is set to $d_i + a_{ij}$. By the inductive hypothesis, $d_i \geq \delta(s, i)$. By the triangle inequality for shortest paths, $\delta(s, j) \leq \delta(s, i) + a_{ij}$. Thus, the new value for d_j satisfies:

$$d_j^{\text{new}} = d_i + a_{ij} \geq \delta(s, i) + a_{ij} \geq \delta(s, j)$$

The property is maintained after the update. □

Theorem 1. *The algorithm terminates.*

Proof. The algorithm terminates when the set OPEN is empty. A node j is added to OPEN only when its label d_j decreases. Therefore, we need to prove that each d_j can only be decreased a finite number of times.

From the previous lemma, we know that $d_j \geq \delta(s, j)$, and d_j can only decrease. If we assume that arc lengths a_{ij} are integers, then the number of possible values for d_j between its initial value and its lower bound $\delta(s, j)$ is finite. Therefore, the label of each node j can only be updated a finite number of times, and thus can only be added to the OPEN set a finite number of times. Since the number of nodes is also finite, the algorithm must terminate. □

Part B: Correctness

We now prove that upon termination, the algorithm finds a shortest path. That is, the final label d_t equals the shortest path distance $\delta(s,t)$.

Theorem 2. *If the graph G contains no negative cycles and there is a path from s to t, the modified label correcting algorithm terminates with $d_t = \delta(s,t)$.*

Proof. We know from Lemma 2 that $d_j \geq \delta(s,j)$ for all nodes j throughout the algorithm. Thus, at termination, $d_t \geq \delta(s,t)$. We only need to prove that $d_t \leq \delta(s,t)$. We prove this by contradiction. Assume the algorithm terminates with $d_t > \delta(s,t)$. Let $P^* = (s = v_0, v_1, \ldots, v_k = t)$ be a shortest path from s to t. Since $d_{v_0} = d_s = 0 = \delta(s,s)$ and we assume $d_{v_k} = d_t > \delta(s,t)$, there must be a first node v_m on this path for which the final label d_{v_m} satisfies $d_{v_m} > \delta(s,v_m)$. Since v_m is the first such node on P^*, the label for the preceding node, v_{m-1}, must be correct at termination. That is, $d_{v_{m-1}} = \delta(s,v_{m-1})$. Because the algorithm has terminated, the set OPEN is empty. Which implies that node v_{m-1} must have been removed from OPEN at some point after its label was last updated. Let's consider the last time v_{m-1} was removed from OPEN. At that time, its label was its final, optimal value, $d_{v_{m-1}} = \delta(s,v_{m-1})$. During the processing of v_{m-1}, the algorithm examined the arc (v_{m-1}, v_m). Since the final label d_{v_m} is not $\delta(s,v_m)$, the relaxation step for v_m via v_{m-1} must have failed. This means the condition for the update was not met:

$$d_{v_{m-1}} + a_{v_{m-1},v_m} \geq \min\{d_{v_m}, \text{UPPER} - u_{v_m}\}$$

(where d_{v_m} and UPPER are the values at that time). Since labels only decrease, the final values also satisfy this inequality. Substituting $d_{v_{m-1}} = \delta(s,v_{m-1})$ and noting that $\delta(s,v_{m-1}) + a_{v_{m-1},v_m} = \delta(s,v_m)$ because (v_{m-1}, v_m) is an arc on a shortest path, we get:

$$\delta(s,v_m) \geq \min\{d_{v_m}, \text{UPPER} - u_{v_m}\}$$

This implies two conditions must hold:

1. $\delta(s,v_m) \geq d_{v_m}$
2. $\delta(s,v_m) \geq \text{UPPER} - u_{v_m}$

Let's analyze the first condition. We already know from Lemma 2 that $d_{v_m} \geq \delta(s,v_m)$. Combining this with $\delta(s,v_m) \geq d_{v_m}$ implies that $d_{v_m} = \delta(s,v_m)$. This contradicts our assumption that v_m was the first node on the path with $d_{v_m} > \delta(s,v_m)$. The pruning condition $\text{UPPER} - u_j$ does not discard the shortest path: UPPER is the length of some $s-t$ path, so $\text{UPPER} \geq \delta(s,t)$. The value u_{v_m} is an underestimate of the distance from v_m to t, so $u_{v_m} \leq \delta(v_m,t)$. Therefore,

$$\text{UPPER} - u_{v_m} \geq \delta(s,t) - \delta(v_m,t)$$

Since v_m is on a shortest path to t, we have $\delta(s,t) = \delta(s,v_m) + \delta(v_m,t)$. Substituting this gives:

$$\text{UPPER} - u_{v_m} \geq (\delta(s,v_m) + \delta(v_m,t)) - \delta(v_m,t) = \delta(s,v_m)$$

This shows that $\delta(s,v_m) \leq \text{UPPER} - u_{v_m}$ is always true. The pruning step only prevents an update if $\delta(s,v_m) \geq \text{UPPER} - u_{v_m}$, which can only happen if equality holds. This pruning is safe because if $d_i + a_{ij} + u_j \geq \text{UPPER}$, the path through (i,j) cannot lead to a better solution than the one that defined the current UPPER.

To this end, we have proved that the assumption that a node on the shortest path has a suboptimal label at termination is false. Thus, all nodes on the shortest path, including t, must have their optimal labels. That is, upon termination, $d_t = \delta(s,t)$. □

2.2 Label Correcting Algorithm as a Special Case

We can see that the Label Correcting Algorithm given in class is a special case of the modified algorithm through comparing their update rules:

- **Standard LCA Update Rule:**

$$d_i + a_{ij} < \min\{d_j, \text{UPPER}\}$$

- **Modified LCA Update Rule:**

$$d_i + a_{ij} < \min\{d_j, \text{UPPER} - u_j\}$$

it's straightforward that setting u_j to zero in modified LCA will lead to standard LCA. But it's also worth noting that setting $u_j = 0$ implies that $\delta(j, t) >= 0$ for any node j, which is always true only when arc length ≥ 0, which is consistent with the conditions for standard LCA.

Problem 3: Object Partition as Shortest Path Problem

We can model this problem as finding the shortest path in a directed acyclic graph (DAG). Let's construct the graph $G = (V, E)$ as follows:

- **Nodes:** We define a set of $N + 1$ nodes, $V = \{v_0, v_1, \ldots, v_N\}$. Each node v_k represents a potential boundary point in our partition. Specifically, node v_k represents making a partition after the k-th object. v_0 represents before the 1st object.
- **Edges:** For every pair (i, j) such that $0 \leq i < j \leq N$, we draw a directed edge from node v_i to node v_j. This edge (v_i, v_j) represents forming a single cluster containing objects $\{i + 1, i + 2, \ldots, j\}$.
- **Edge Weights:** The weight of edge (v_i, v_j), denoted as $w(v_i, v_j) = a_{i+1,j}$, is the cost of generating the corresponding cluster.
- **Node Distance:** The distance of node v_i, denoted as d_{v_i}, can be seen as the minimum cost to generate a valid partition for objects 1 i. Thus, the distance of node v_N is the minimum cost for partitioning all objects.

Therefore, the problem of finding the minimum cost grouping is equivalent to finding the **shortest path** from node v_0 to node v_N in the constructed graph G. We can either solve it using any shortest path algorithm, or in a DP style. For the latter, the state transition is:

$$d_{v_i} = \min_{0 \leq j < i} \{d_{v_j} + a_{j+1,i}\}$$

and $d_{v_0} = 0$. Iteration over all nodes v_1 v_N will give the minimum cost for partitioning all objects. To recover the actual grouping, one can record the predecessor for each node in shortest-path solution, or record the $\arg\min_j \{d_{v_j} + a_{j+1,i}\}$ for each i during Iteration.

Algorithm 1 Optimal Clustering DP

```
Input: Number of objects N, and costs a_ij for 1 ≤ i ≤ j ≤ N.
Output: The minimum total cost and the corresponding partition.
Initialize arrays C[0...N] and P[1...N].
C[0] ← 0
for j ← 1 to N do
    C[j] ← ∞
    for i ← 0 to j − 1 do
        if C[i] + a_{i+1,j} < C[j] then
            C[j] ← C[i] + a_{i+1,j}
            P[j] ← i
        end if
    end for
end for
{The minimum cost is C[N]. Now, reconstruct the clusters.}
Initialize an empty list of clusters.
j ← N
while j > 0 do
    i ← P[j]
    Add cluster {i + 1, ..., j} to the clusters.
    j ← i
end while
return C[N] and clusters.
```

Problem 4:

Problem 5:

Problem 6:

Problem 7:

Problem 8:

Problem 9: